

LOGICAL STRUCTURES

Lecture 4. Predicate logic

Advanced Program in Computer Science

Spring, 2025

Contents

- ❶ The need for a richer language
- ❷ Predicate logic as a formal language
 - Terms
 - Formulas
 - Free and bound variables
 - Substitution

1. The need for a richer language

- Propositional logic dealt satisfactorily with sentence components like: **not**, **and**, **or**, and **if...then**.

1. The need for a richer language

- Propositional logic dealt satisfactorily with sentence components like: **not**, **and**, **or**, and **if...then**.
- But how about **there exists...**, **all...**, **among...**, and **only...**?

1. The need for a richer language

- Propositional logic dealt satisfactorily with sentence components like: **not**, **and**, **or**, and **if...then**.
- But how about **there exists...**, **all...**, **among...**, and **only...**?
- The need led to the design of *predicate logic*, also called *first-order logic*.

Let us consider the declarative sentence

Every student x is younger than **some** instructor y .

1. The need for a richer language

- Propositional logic dealt satisfactorily with sentence components like: **not**, **and**, **or**, and **if...then**.
- But how about **there exists...**, **all...**, **among...**, and **only...**?
- The need led to the design of *predicate logic*, also called *first-order logic*.

Let us consider the declarative sentence

Every student x is younger than **some** instructor y .

In propositional logic, we could identify this assertion with a propositional atom p .

1. The need for a richer language

- Propositional logic dealt satisfactorily with sentence components like: **not**, **and**, **or**, and **if...then**.
- But how about **there exists...**, **all...**, **among...**, and **only...**?
- The need led to the design of *predicate logic*, also called *first-order logic*.

Let us consider the declarative sentence

Every student x is younger than **some** instructor y .

In propositional logic, we could identify this assertion with a propositional atom p . However, that fails to reflect the finer logical structure of this sentence.

Predicates

Every student x is younger than **some** instructor y .

Predicates

Every student x is younger than **some** instructor y .

Example 2.1

- $S(x) :$ x is a student

Predicates

Every student x is younger than **some** instructor y .

Example 2.1

- $S(x) :$ x is a student
- $I(x) :$ x is an instructor

Predicates

Every student x is younger than **some** instructor y .

Example 2.1

- $S(x)$: x is a student
- $I(x)$: x is an instructor
- $Y(x, y)$: x is younger than y

Predicates

Every student x is younger than **some** instructor y .

Example 2.1

- $S(x)$: x is a student
- $I(x)$: x is an instructor
- $Y(x, y)$: x is younger than y

S, I, Y are *predicates* and x, y are *variables*.

Predicates

Every student x is younger than **some** instructor y .

Example 2.1

- $S(x)$: x is a student
- $I(x)$: x is an instructor
- $Y(x, y)$: x is younger than y

S, I, Y are *predicates* and x, y are *variables*.

- $S(\text{Minh})$: Minh is a student

Predicates

Every student x is younger than **some** instructor y .

Example 2.1

- $S(x)$: x is a student
- $I(x)$: x is an instructor
- $Y(x, y)$: x is younger than y

S, I, Y are *predicates* and x, y are *variables*.

- $S(\text{Minh})$: Minh is a student
- $I(\text{Luyen})$: Luyen is an instructor

Predicates

Every student x is younger than **some** instructor y .

Example 2.1

- $S(x)$: x is a student
- $I(x)$: x is an instructor
- $Y(x, y)$: x is younger than y

S, I, Y are *predicates* and x, y are *variables*.

- $S(\text{Minh})$: Minh is a student
- $I(\text{Luyen})$: Luyen is an instructor
- $Y(\text{Minh}, \text{Luyen})$: Minh is younger than Luyen.

Predicates

Every student x is younger than **some** instructor y .

Example 2.1

- $S(x)$: x is a student
- $I(x)$: x is an instructor
- $Y(x, y)$: x is younger than y

S, I, Y are *predicates* and x, y are *variables*.

- $S(\text{Minh})$: Minh is a student
- $I(\text{Luyen})$: Luyen is an instructor
- $Y(\text{Minh}, \text{Luyen})$: Minh is younger than Luyen.

$$S(x) \rightarrow (I(y) \wedge Y(x, y))$$

Predicates

Every student x is younger than **some** instructor y .

Example 2.1

- $S(x)$: x is a student
- $I(x)$: x is an instructor
- $Y(x, y)$: x is younger than y

S, I, Y are *predicates* and x, y are *variables*.

- $S(\text{Minh})$: Minh is a student
- $I(\text{Luyen})$: Luyen is an instructor
- $Y(\text{Minh}, \text{Luyen})$: Minh is younger than Luyen.

$$S(x) \rightarrow (I(y) \wedge Y(x, y))$$

Q: every, some?

Predicates

Every student x is younger than **some** instructor y .

Example 2.1

- $S(x)$: x is a student
- $I(x)$: x is an instructor
- $Y(x, y)$: x is younger than y

S, I, Y are *predicates* and x, y are *variables*.

- $S(\text{Minh})$: Minh is a student
- $I(\text{Luyen})$: Luyen is an instructor
- $Y(\text{Minh}, \text{Luyen})$: Minh is younger than Luyen.

$$S(x) \rightarrow (I(y) \wedge Y(x, y))$$

Q: **every, some**? Quantifiers.

Quantifiers

- $\forall$: *for all*

Quantifiers

- $\forall$: *for all*
- $\exists$: *there exists* or *for some*

Quantifiers

- $\forall$: *for all*
- $\exists$: *there exists* or *for some*

Sentence:

Every student x is younger than **some** instructor y .

Quantifiers

- $\forall$: *for all*
- $\exists$: *there exists* or *for some*

Sentence:

Every student x is younger than **some** instructor y .

Predicate logic:

$$\forall x (S(x) \rightarrow (\exists y (I(y) \wedge Y(x, y))))$$

Quantifiers

- $\forall$: *for all*
- $\exists$: *there exists* or *for some*

Sentence:

Every student x is younger than **some** instructor y .

Predicate logic:

$$\forall x (S(x) \rightarrow (\exists y (I(y) \wedge Y(x, y))))$$

In this example, the re-translation results in

For every x , if x is a student, then there is some y which is an instructor such that x is younger than y .

Example

Example 2.2

Sentence: ‘*Not all birds can fly.*’

Example

Example 2.2

Sentence: '*Not all birds can fly.*'

- $B(x)$: x is a bird
- $F(x)$: x can fly

Example

Example 2.2

Sentence: ‘*Not all birds can fly.*’

- $B(x)$: x is a bird
- $F(x)$: x can fly

Predicate logic:

$$\neg(\forall x(B(x) \rightarrow F(x)))$$

Example

Example 2.2

Sentence: ‘*Not all birds can fly.*’

- $B(x)$: x is a bird
- $F(x)$: x can fly

Predicate logic:

$$\neg(\forall x(B(x) \rightarrow F(x)))$$

We could code this as

$$\exists x(B(x) \wedge \neg F(x))$$

Example

Example 2.2

Sentence: ‘*Not all birds can fly.*’

- $B(x)$: x is a bird
- $F(x)$: x can fly

Predicate logic:

$$\neg(\forall x(B(x) \rightarrow F(x)))$$

We could code this as

$$\exists x(B(x) \wedge \neg F(x))$$

It means

There is some x which is a bird and cannot fly.

Natural deduction vs Semantics

- We will extend **natural deduction rules** to work with predicate logic.

Natural deduction vs Semantics

- We will extend **natural deduction rules** to work with predicate logic.
- We will explain about **semantically entailment** and the process of **evaluating a predicate formula**

Natural deduction vs Semantics

- We will extend **natural deduction rules** to work with predicate logic.
- We will explain about **semantically entailment** and the process of **evaluating a predicate formula**
- natural deduction calculus for predicate logic is sound and complete with respect to semantic entailment

Natural deduction vs Semantics

- We will extend **natural deduction rules** to work with predicate logic.
- We will explain about **semantically entailment** and the process of **evaluating a predicate formula**
- natural deduction calculus for predicate logic is sound and complete with respect to semantic entailment

$$\boxed{\phi_1, \phi_2, \dots, \phi_n \vdash \psi \quad \text{iff} \quad \phi_1, \phi_2, \dots, \phi_n \models \psi}$$

Natural deduction vs Semantics

‘No books are gaseous. Dictionaries are books. Therefore, no dictionary is gaseous.’

Natural deduction vs Semantics

‘No books are gaseous. Dictionaries are books. Therefore, no dictionary is gaseous.’

The predicates we choose are

- $B(x)$: x is a book
- $G(x)$: x is gaseous
- $D(x)$: x is a dictionary.

Natural deduction vs Semantics

‘No books are gaseous. Dictionaries are books. Therefore, no dictionary is gaseous.’

The predicates we choose are

- $B(x)$: x is a book
- $G(x)$: x is gaseous
- $D(x)$: x is a dictionary.

Then

- Sequent:

$$\neg\exists x (B(x) \wedge G(x)), \forall x (D(x) \rightarrow B(x)) \vdash \neg\exists x (D(x) \wedge G(x))$$

Natural deduction vs Semantics

‘No books are gaseous. Dictionaries are books. Therefore, no dictionary is gaseous.’

The predicates we choose are

- $B(x)$: x is a book
- $G(x)$: x is gaseous
- $D(x)$: x is a dictionary.

Then

- Sequent:

$$\neg\exists x (B(x) \wedge G(x)), \forall x (D(x) \rightarrow B(x)) \vdash \neg\exists x (D(x) \wedge G(x))$$

- Semantics:

$$\neg\exists x (B(x) \wedge G(x)), \forall x (D(x) \rightarrow B(x)) \models \neg\exists x (D(x) \wedge G(x))$$

Function symbols

Example 2.3

Express the following sentence using predicates

‘Every child is younger than its mother.’

Function symbols

Example 2.3

Express the following sentence using predicates

‘Every child is younger than its mother.’

- $C(x)$: x is a child
- $M(x, y)$: x is y ’s mother
- $Y(x, y)$: x is younger than y .

Function symbols

Example 2.3

Express the following sentence using predicates

‘Every child is younger than its mother.’

- $C(x)$: x is a child
- $M(x, y)$: x is y ’s mother
- $Y(x, y)$: x is younger than y .

Then

$$\forall x \forall y (C(x) \wedge M(y, x) \rightarrow Y(x, y))$$

Function symbols

Example 2.4

Express the following sentence using predicates

‘Andy and Paul have the same maternal grandmother.’

Function symbols

Example 2.4

Express the following sentence using predicates

‘Andy and Paul have the same maternal grandmother.’

$$\forall x \forall y \forall u \forall v (M(x, y) \wedge M(y, a) \wedge M(u, v) \wedge M(v, p) \rightarrow x = u)$$

Function symbols

Example 2.4

Express the following sentence using predicates

‘Andy and Paul have the same maternal grandmother.’

$$\forall x \forall y \forall u \forall v (M(x, y) \wedge M(y, a) \wedge M(u, v) \wedge M(v, p) \rightarrow x = u)$$

Too complicated for a simple sentence $\Rightarrow$ use *function symbols*.

Function symbols

- $C(x)$: x is a child
- $m(x)$ is the mother of x

Function symbols

- $C(x)$: x is a child
- $m(x)$ is the mother of x

‘Every child is younger than its mother.’

$$\forall x (C(x) \rightarrow Y(m, m(x)))$$

Function symbols

- $C(x)$: x is a child
- $m(x)$ is the mother of x

‘Every child is younger than its mother.’

$$\forall x (C(x) \rightarrow Y(m, m(x)))$$

‘Andy and Paul have the same maternal grandmother.’

$$m(m(a)) = m(m(p))$$

where a is Andy and p is Paul.

Examples

Example 2.5

Express the following sentence using predicates and function symbols (if needed)

'Ann likes one of Mary's brothers'

How if Ann likes all of Mary's brother?

Examples

Example 2.5

Express the following sentence using predicates and function symbols (if needed)

'Ann likes one of Mary's brothers'

How if Ann likes all of Mary's brother?

- $B(x, y) : x$ is a brother of y

Examples

Example 2.5

Express the following sentence using predicates and function symbols (if needed)

'Ann likes one of Mary's brothers'

How if Ann likes all of Mary's brother?

- $B(x, y) : x$ is a brother of y
- $L(x, y) : x$ likes y

Examples

Example 2.5

Express the following sentence using predicates and function symbols (if needed)

'Ann likes one of Mary's brothers'

How if Ann likes all of Mary's brother?

- $B(x, y) : x$ is a brother of y
- $L(x, y) : x$ likes y

Then

$$\exists x (B(x, m) \wedge L(a, x))$$

Examples

Example 2.5

Express the following sentence using predicates and function symbols (if needed)

'Ann likes one of Mary's brothers'

How if Ann likes all of Mary's brother?

- $B(x, y) : x$ is a brother of y
- $L(x, y) : x$ likes y

Then

$$\exists x (B(x, m) \wedge L(a, x))$$

$$\forall x (B(x, m) \rightarrow L(a, x))$$

Exercise 1.

Use the predicates

$A(x, y)$: x admires y

$B(x, y)$: x attended y

$P(x)$: x is a professor

$S(x)$: x is a student

$L(x)$: x is a lecture

m : Mary

to translate the following into predicate logic:

- a) Mary admires every professor.
- b) Some professor admires Mary.
- c) Mary admires herself.
- d) No student attended every lecture.
- e) No lecture was attended by every student.
- f) No lecture was attended by any student.

Exercise 2.

Find appropriate predicates and their specification to translate the following into predicate logic:

- a) All red things are in the box.
- b) Only red things are in the box.
- c) No animal is both a cat and a dog.
- d) Every prize was won by a boy.
- e) A boy won every prize.

2. Predicate logic as a formal language

- individuals such as a and p (referring to Andy and Paul) are *examples* or *constants*

2. Predicate logic as a formal language

- individuals such as a and p (referring to Andy and Paul) are *examples* or *constants*
- x and v are *variables*

2. Predicate logic as a formal language

- individuals such as a and p (referring to Andy and Paul) are *examples* or *constants*
- x and v are *variables*
- *function symbols* refer to objects such as $m(a)$ and $g(x, y)$

2. Predicate logic as a formal language

- individuals such as a and p (referring to Andy and Paul) are *examples* or *constants*
- x and v are *variables*
- *function symbols* refer to objects such as $m(a)$ and $g(x, y)$
- expressions in predicate logic which denotes objects are called *term*

2. Predicate logic as a formal language

- individuals such as a and p (referring to Andy and Paul) are *examples* or *constants*
- x and v are *variables*
- *function symbols* refer to objects such as $m(a)$ and $g(x, y)$
- expressions in predicate logic which denotes objects are called *term*
- The other sort of things in predicate logic denotes truth values; expressions of this kind are *formulas*: $Y(x, m(x))$ is a formula, though x and $m(x)$ are term.

2. Predicate logic as a formal language

- individuals such as a and p (referring to Andy and Paul) are *examples* or *constants*
- x and v are *variables*
- *function symbols* refer to objects such as $m(a)$ and $g(x, y)$
- expressions in predicate logic which denotes objects are called *term*
- The other sort of things in predicate logic denotes truth values; expressions of this kind are *formulas*: $Y(x, m(x))$ is a formula, though x and $m(x)$ are term.
- A *predicate vocabulary* consists of three sets: a set of predicate symbols $\mathcal{P}$, a set of function symbols $\mathcal{F}$ and a set of constant symbols $\mathcal{C}$. For convenient, drop $\mathcal{C}$ since they can be seen as functions which don't take any arguments.

Terms

Definition 3.1

Terms are defined as follows

- Any variable is a term
- if $c \in \mathcal{F}$ is a *nullary function* (no arguments then is a constant), then c is a term
- if $t_1, t_2, \dots, t_n$ are terms and $f \in \mathcal{F}$ has *arity* (number of arguments) $n > 0$, then $f(t_1, t_2, \dots, t_n)$ is a term
- nothing else is a term.

Terms

Definition 3.1

Terms are defined as follows

- Any variable is a term
- if $c \in \mathcal{F}$ is a *nullary function* (no arguments then is a constant), then c is a term
- if $t_1, t_2, \dots, t_n$ are terms and $f \in \mathcal{F}$ has *arity* (number of arguments) $n > 0$, then $f(t_1, t_2, \dots, t_n)$ is a term
- nothing else is a term.

Backus Naur form: $t ::= x \mid c \mid f(t, t, \dots, t)$

where x ranges over a set of variables **var**, c over nullary function symbols in $\mathcal{F}$, and f over those elements of $\mathcal{F}$ with arity $n > 0$.

Terms

Example 3.2

Suppose that n , f and g are function symbols, respectively nullary, unary and binary.

Terms

Example 3.2

Suppose that n , f and g are function symbols, respectively nullary, unary and binary.

- $g(f(n), n)$ and $f(g(n, f(n)))$ are terms

Terms

Example 3.2

Suppose that n , f and g are function symbols, respectively nullary, unary and binary.

- $g(f(n), n)$ and $f(g(n, f(n)))$ are terms
- $g(n)$ and $f(f(n), n)$ are not terms

Formulas

Definition 3.3

We define the set of formulas over $(\mathcal{F}, \mathcal{P})$ inductively, using the already defined set of terms over $\mathcal{F}$:

- if $P \in \mathcal{P}$ is a predicate symbol of arity $n \geq 1$, and if $t_1, t_2, \dots, t_n$ are terms over $\mathcal{F}$, then $P(t_1, t_2, \dots, t_n)$ is a formula.
- if ϕ is a formula, then so is $(\neg\phi)$
- if ϕ and ψ are formulas, then so are $(\phi \wedge \psi)$, $(\phi \vee \psi)$ and $(\phi \rightarrow \psi)$
- if ϕ is a formula and x is a variable, then $(\forall x\phi)$ and $(\exists x\phi)$ are formulas
- nothing else is a formula.

Backus Naur form:

$$\phi ::= P(t_1, t_2, \dots, t_n) \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi) \mid (\forall x \phi) \mid (\exists x \phi)$$

Formulas

Convention

For convenience, we retain the usual binding priorities agreed upon in Convention in **Lecture 1** and add that $\forall y$ and $\exists y$ bind like $\neg$. Thus, the order is:

- $\neg, \forall y$ and $\exists y$ bind most tightly;
- then $\vee$ and $\wedge$;
- then $\rightarrow$, which is *right-associative*: $a \rightarrow b \rightarrow c \equiv a \rightarrow (b \rightarrow c)$

Example

Example 3.4

Translate the sentence: '*Every son of my father is my brother.*' into predicate logic.

Example

Example 3.4

Translate the sentence: '*Every son of my father is my brother.*' into predicate logic.

- choose a constant m for '*me*' or '*I*'.

Example

Example 3.4

Translate the sentence: '*Every son of my father is my brother.*' into predicate logic.

- choose a constant m for '*me*' or '*I*'.
- choose $\{S, F, B\}$ as the set of predicates with meaning
 - $S(x, y)$: x is a son of y
 - $F(x, y)$: x is the father of y
 - $B(x, y)$: x is a brother of y

Example

Example 3.4

Translate the sentence: '*Every son of my father is my brother.*' into predicate logic.

- choose a constant m for '*me*' or '*I*'.
- choose $\{S, F, B\}$ as the set of predicates with meaning
 - $S(x, y)$: x is a son of y
 - $F(x, y)$: x is the father of y
 - $B(x, y)$: x is a brother of y
- two options for representing '*father*' as a predicate or a function symbol.

Example

‘Every son of my father is my brother.’

- ① represent *‘father’* as a predicate $F(x, y) : x$ is the father of y

Example

‘Every son of my father is my brother.’

- ❶ represent *‘father’* as a predicate $F(x, y) : x$ is the father of y

$$\forall x \forall y (F(x, m) \wedge S(y, x) \rightarrow B(y, m))$$

Example

‘Every son of my father is my brother.’

- ❶ represent *‘father’* as a predicate $F(x, y) : x$ is the father of y

$$\forall x \forall y (F(x, m) \wedge S(y, x) \rightarrow B(y, m))$$

- ❷ represent *‘father’* as a symbol function: $f(x)$ is the father of x .

Example

‘Every son of my father is my brother.’

- ❶ represent *‘father’* as a predicate $F(x, y) : x$ is the father of y

$$\forall x \forall y (F(x, m) \wedge S(y, x) \rightarrow B(y, m))$$

- ❷ represent *‘father’* as a symbol function: $f(x)$ is the father of x .

$$\forall x (S(x, f(m)) \rightarrow B(x, m))$$

Parse tree

To draw parse tree, do the same way as for propositional formulas but with two additional sorts of nodes:

Parse tree

To draw parse tree, do the same way as for propositional formulas but with two additional sorts of nodes:

- the quantifiers $\forall x$ and $\exists y$ forms nodes and have, like negation, just one subtree

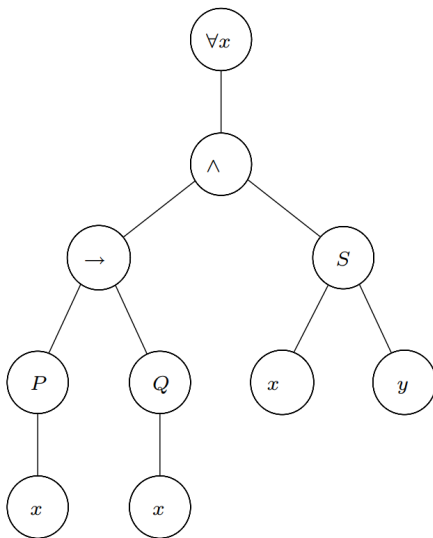
Parse tree

To draw parse tree, do the same way as for propositional formulas but with two additional sorts of nodes:

- the quantifiers $\forall x$ and $\exists y$ forms nodes and have, like negation, just one subtree
- predicate expressions, $P(t_1, t_2, \dots, t_n)$ have the symbol P as a node, but now P has n many subtrees, namely the parse trees of the terms $t_1, t_2, \dots, t_n$

Draw the parse tree of $\forall x ((P(x) \rightarrow Q(x)) \wedge S(x, y))$

Draw the parse tree of $\forall x ((P(x) \rightarrow Q(x)) \wedge S(x, y))$



Exercise

Draw the parse tree of

a. $\exists y \forall z (\exists x Q(x, z) \rightarrow (P(x, z) \wedge R(y, z)))$

b. $(\forall x P(x, z) \rightarrow \exists t Q(x, t)) \vee \neg(R(y) \rightarrow \exists y S(y, t))$

Free and bound variables

In a parse tree, variables occur at two different sorts of places

- next to quantifiers $\forall$ and $\exists$
- leaf nodes

Free and bound variables

In a parse tree, variables occur at two different sorts of places

- next to quantifiers $\forall$ and $\exists$
- leaf nodes

By observing the previous parse tree, there are two principal such occurrences:

- three leaf nodes x .

Free and bound variables

In a parse tree, variables occur at two different sorts of places

- next to quantifiers $\forall$ and $\exists$
- leaf nodes

By observing the previous parse tree, there are two principal such occurrences:

- three leaf nodes x . If we walk up the tree beginning at any one of these x leaves, we run into the quantifier $\forall x$.

Free and bound variables

In a parse tree, variables occur at two different sorts of places

- next to quantifiers $\forall$ and $\exists$
- leaf nodes

By observing the previous parse tree, there are two principal such occurrences:

- three leaf nodes x . If we walk up the tree beginning at any one of these x leaves, we run into the quantifier $\forall x$. This means that those occurrences of x are *bound* to $\forall x$;

Free and bound variables

In a parse tree, variables occur at two different sorts of places

- next to quantifiers $\forall$ and $\exists$
- leaf nodes

By observing the previous parse tree, there are two principal such occurrences:

- three leaf nodes x . If we walk up the tree beginning at any one of these x leaves, we run into the quantifier $\forall x$. This means that those occurrences of x are *bound* to $\forall x$;
- beginning at the only y leaf, we run into the only quantifier which is $\forall x$,

Free and bound variables

In a parse tree, variables occur at two different sorts of places

- next to quantifiers $\forall$ and $\exists$
- leaf nodes

By observing the previous parse tree, there are two principal such occurrences:

- three leaf nodes x . If we walk up the tree beginning at any one of these x leaves, we run into the quantifier $\forall x$. This means that those occurrences of x are *bound* to $\forall x$;
- beginning at the only y leaf, we run into the only quantifier which is $\forall x$, so y is *free* in this formula.

Free and bound variables

Definition 3.5

Let ϕ be a formula in predicate logic.

Free and bound variables

Definition 3.5

Let ϕ be a formula in predicate logic. An occurrence of x in ϕ is *free* in ϕ

Free and bound variables

Definition 3.5

Let ϕ be a formula in predicate logic. An occurrence of x in ϕ is *free* in ϕ if it is a leaf node in the parse tree of ϕ such that there is no path upwards from that node x to a node $\forall x$ or $\exists x$.

Free and bound variables

Definition 3.5

Let ϕ be a formula in predicate logic. An occurrence of x in ϕ is *free* in ϕ if it is a leaf node in the parse tree of ϕ such that there is no path upwards from that node x to a node $\forall x$ or $\exists x$. Otherwise, that occurrence of x is called *bound*.

Free and bound variables

Definition 3.5

Let ϕ be a formula in predicate logic. An occurrence of x in ϕ is *free* in ϕ if it is a leaf node in the parse tree of ϕ such that there is no path upwards from that node x to a node $\forall x$ or $\exists x$. Otherwise, that occurrence of x is called *bound*.

For $\forall x\phi$, or $\exists x\phi$, we say that ϕ — minus any of ϕ 's subformulas $\exists x\psi$ or $\forall x\psi$ — is the *scope* of $\forall x$, respectively $\exists x$.

Free and bound variables

Definition 3.5

Let ϕ be a formula in predicate logic. An occurrence of x in ϕ is **free** in ϕ if it is a leaf node in the parse tree of ϕ such that there is no path upwards from that node x to a node $\forall x$ or $\exists x$. Otherwise, that occurrence of x is called **bound**.

For $\forall x\phi$, or $\exists x\phi$, we say that ϕ — minus any of ϕ 's subformulas $\exists x\psi$ or $\forall x\psi$ — is the **scope** of $\forall x$, respectively $\exists x$.

Thus, if x occurs in ϕ , then it is bound if, and only if, it is in the scope of some $\exists x$ or some $\forall x$; otherwise it is free

Free and bound variables

Definition 3.5

Let ϕ be a formula in predicate logic. An occurrence of x in ϕ is **free** in ϕ if it is a leaf node in the parse tree of ϕ such that there is no path upwards from that node x to a node $\forall x$ or $\exists x$. Otherwise, that occurrence of x is called **bound**.

For $\forall x\phi$, or $\exists x\phi$, we say that ϕ — minus any of ϕ 's subformulas $\exists x\psi$ or $\forall x\psi$ — is the **scope** of $\forall x$, respectively $\exists x$.

Thus, if x occurs in ϕ , then it is bound if, and only if, it is in the scope of some $\exists x$ or some $\forall x$; otherwise it is free

The scope of $\forall x$ in $\forall x(P(x) \rightarrow \exists xQ(x))$ is

Free and bound variables

Definition 3.5

Let ϕ be a formula in predicate logic. An occurrence of x in ϕ is **free** in ϕ if it is a leaf node in the parse tree of ϕ such that there is no path upwards from that node x to a node $\forall x$ or $\exists x$. Otherwise, that occurrence of x is called **bound**.

For $\forall x\phi$, or $\exists x\phi$, we say that ϕ — minus any of ϕ 's subformulas $\exists x\psi$ or $\forall x\psi$ — is the **scope** of $\forall x$, respectively $\exists x$.

Thus, if x occurs in ϕ , then it is bound if, and only if, it is in the scope of some $\exists x$ or some $\forall x$; otherwise it is free

The scope of $\forall x$ in $\forall x(P(x) \rightarrow \exists xQ(x))$ is $P(x)$.

Free and bound variables

Example 3.6

Consider the formula $(\forall x(P(x) \wedge Q(x))) \rightarrow (\neg P(x) \vee Q(y))$. Identify free and bound variables.

Free and bound variables

Example 3.6

Consider the formula $(\forall x(P(x) \wedge Q(x))) \rightarrow (\neg P(x) \vee Q(y))$. Identify free and bound variables.

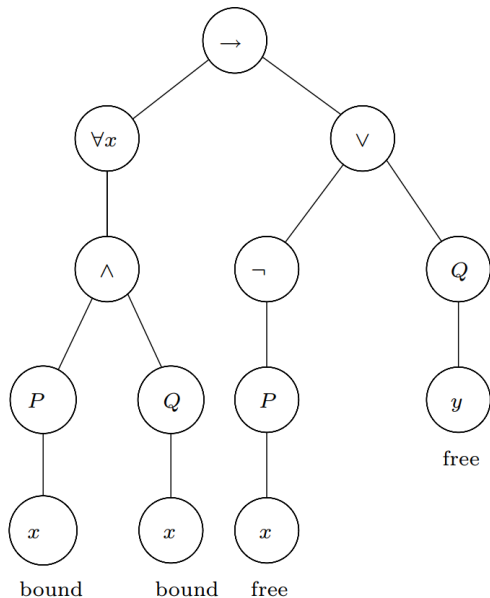
- **Step 1:** Draw parse tree of the formula

Free and bound variables

Example 3.6

Consider the formula $(\forall x(P(x) \wedge Q(x))) \rightarrow (\neg P(x) \vee Q(y))$. Identify free and bound variables.

- **Step 1:** Draw parse tree of the formula
- **Step 2:** From each leaf node, walk up to see if encounter any quantifier



Exercise

Identify all bound and free variable leaves in

a. $\exists y \forall z (\exists x Q(x, z) \rightarrow (P(x, z) \wedge R(y, z)))$

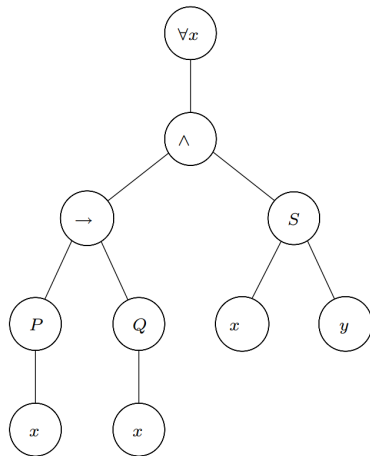
b. $(\forall x P(x, z) \rightarrow \exists t Q(x, t)) \vee \neg(R(y) \rightarrow \exists y S(y, t))$

Substitution

Definition 3.7

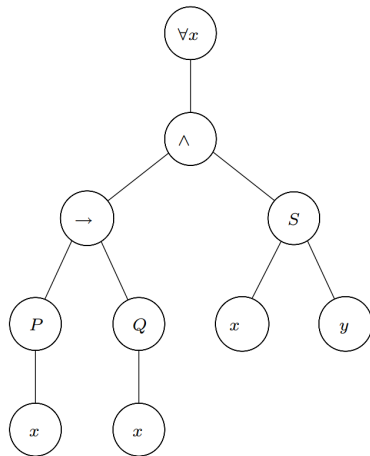
Given a variable x , a term t and a formula ϕ we define $\phi(t/x)$ to be the formula obtained by **replacing** each **free occurrence** of variable x in ϕ with t .

Examples



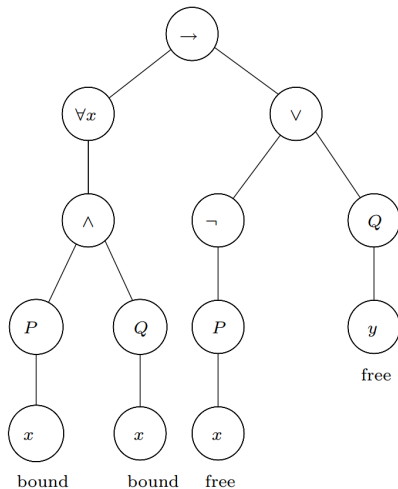
$f(x, y)$ is a term and $\phi[f(x, y)/x] = \phi$

Examples



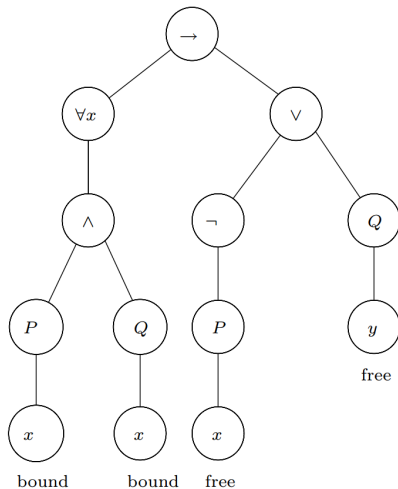
$f(x, y)$ is a term and $\phi[f(x, y)/x] = \phi$ since there is no occurrence of x is free.

Examples



$$\phi = (\forall x(P(x) \wedge Q(x))) \rightarrow (\neg P(\textcolor{red}{x}) \vee Q(y)).$$

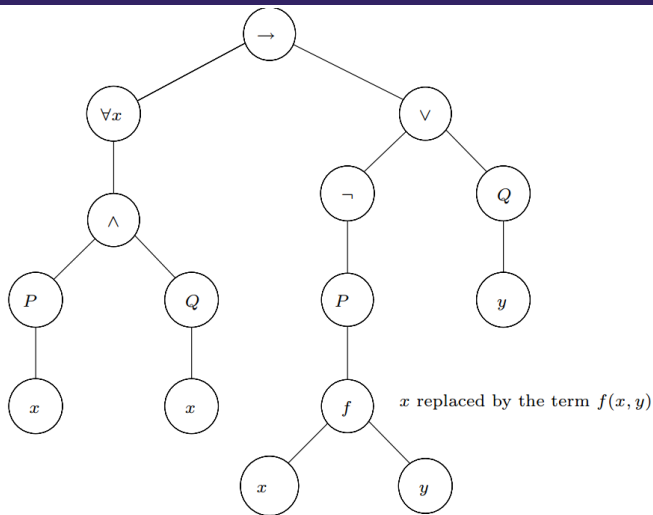
Examples



$$\phi = (\forall x(P(x) \wedge Q(x))) \rightarrow (\neg P(\textcolor{red}{x}) \vee Q(y)).$$

There is only one occurrence of x is free.

Examples



$$\phi[f(x,y)/x] = (\forall x(P(x) \wedge Q(x))) \rightarrow (\neg P(\mathbf{f(x,y)}) \vee Q(y))$$

Example 3.8

- $\forall x P(x) \wedge Q(x) \ [y/x] \quad =$

Example 3.8

- $\forall x P(x) \wedge Q(x) \ [y/x] \quad = \quad \forall x P(x) \wedge Q(y)$

Example 3.8

- $\forall x P(x) \wedge Q(x) \ [y/x] = \forall x P(x) \wedge Q(y)$
- $\exists y R(x, y) \ [z/x] =$

Example 3.8

- $\forall x P(x) \wedge Q(x) \ [y/x] = \forall x P(x) \wedge Q(y)$
- $\exists y R(x, y) \ [z/x] = \exists y R(z, y)$

Example 3.8

- $\forall x P(x) \wedge Q(x) \ [y/x] = \forall x P(x) \wedge Q(y)$
- $\exists y R(x, y) \ [z/x] = \exists y R(z, y)$
- $\exists y R(x, y) \ [z/y] =$

Example 3.8

- $\forall x P(x) \wedge Q(x) \ [y/x] = \forall x P(x) \wedge Q(y)$
- $\exists y R(x, y) \ [z/x] = \exists y R(z, y)$
- $\exists y R(x, y) \ [z/y] = \exists y R(x, y)$

Example 3.8

- $\forall x P(x) \wedge Q(x) \ [y/x] = \forall x P(x) \wedge Q(y)$
- $\exists y R(x, y) \ [z/x] = \exists y R(z, y)$
- $\exists y R(x, y) \ [z/y] = \exists y R(x, y)$
- $\exists y R(x, y) \ [y/x] =$

Example 3.8

- $\forall x P(x) \wedge Q(x) \ [y/x] = \forall x P(x) \wedge Q(y)$
- $\exists y R(x, y) \ [z/x] = \exists y R(z, y)$
- $\exists y R(x, y) \ [z/y] = \exists y R(x, y)$
- $\exists y R(x, y) \ [y/x] = \exists y R(y, y)$

Substitutions and Side effects

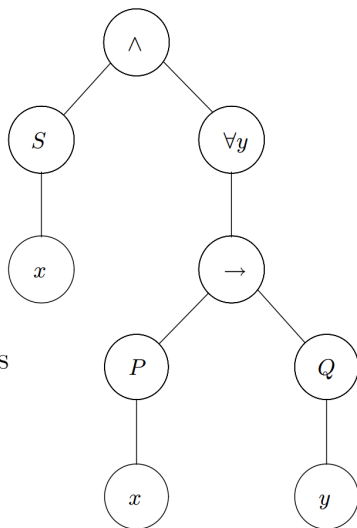
- the term t may contain variable y , where free occurrences of x in ϕ are under the scope of $\exists y$ or $\forall y$ in ϕ .
- by carrying out the substitution $\phi[t/x]$, the value of y gets caught in the scope of $\exists y$ or $\forall y$ and might have been fixed.
- undesired variable captures are to be avoided at all costs

Free term

Definition 3.9

Given a term t , a variable x and a formula ϕ , we say that t is *free for* x in ϕ if no free x leaf in ϕ occurs in the scope of $\forall y$ or $\exists y$ for any variable y occurring in t .

Example



the term $f(y, y)$ is
not free for x in
this formula

Exercise

Let ϕ be $\exists x(P(y, z) \wedge (\forall y(\neg Q(y, x) \vee P(y, z))))$, where P and Q are predicate symbols with two arguments.

- (a) Draw the parse tree of ϕ .
- (b) Identify all bound and free variable leaves in ϕ .
- (c) Is there a variable in ϕ which has free and bound occurrences?
- (d) Consider the terms w (w is a variable), $f(x)$ and $g(y, z)$, where f and g are function symbols with arity 1 and 2, respectively.
 - i. Compute $\phi[w/x]$, $\phi[w/y]$, $\phi[f(x)/y]$ and $\phi[g(y, z)/z]$.
 - ii. Which of w , $f(x)$ and $g(y, z)$ are free for x in ϕ ?
 - iii. Which of w , $f(x)$ and $g(y, z)$ are free for y in ϕ ?
- (e) What is the scope of $\exists x$ in ϕ ?
- (f) Suppose that we change ϕ to $\exists x(P(y, z) \wedge (\forall x(\neg Q(x, x) \vee P(x, z))))$. What is the scope of $\exists x$ now?